16-833: Robot Localization and Mapping, Spring 2026

Homework 4
Dense SLAM with Point-based Fusion

Po-Lin Chen
polinc@andrew.cmu.edu

2. Iterative Closest Point (ICP)

2.1 Projective data association

To set up a valid correspondence, the projected point must lie **within the boundaries** of the vertex map and have a **valid positive depth.** Therefore, u, v, and d must satisfy the following conditions: $0 \leq u < W$**,** $0 \leq v < H$**,** $d \geq 0$**.**

By filtering correspondences by a distance threshold, **the outliers from the correspondence can be removed** to avoid corrupting the computed transformation.

2.2 Linearization

Based on the point-to-plane error formulation, the original non-linear error function is:

$$r_i(\delta R, \delta t) = n_{q_i}^\top((\delta R)p_i' + \delta t - q_i) \tag{1}$$

Using the small-angle assumption, we can parameterize the rotation matrix as $\delta R \approx I + [\omega]_\times$, where the vector $\omega = [\alpha, \beta, \gamma]^\top$. Substituting this into the error function yields:

$$r_i = n_{q_i}^\top(p_i' + [\omega]_\times p_i' + \delta t - q_i) \tag{2}$$

By applying the cross-product property $[\omega]_\times p_i' = -[p_i']_\times \omega$, we can separate the variables to linearize the function:

$$r_i = -n_{q_i}^\top[p_i']_\times \omega + n_{q_i}^\top \delta t + n_{q_i}^\top(p_i' - q_i) \tag{3}$$

By reorganizing these terms into the state vector $x = [\alpha, \beta, \gamma, t_x, t_y, t_z]^\top$, we can extract A_i and b_i:

Matrix Form:

$$A_i = \begin{bmatrix} -n_{q_i}^\top[p_i']_\times & n_{q_i}^\top \end{bmatrix} \tag{4}$$

$$b_i = n_{q_i}^\top(p_i' - q_i) \tag{5}$$

Component Form (Expanded):

$$A_i = \begin{bmatrix} -n_{q_{i,y}}p_{i,z}' + n_{q_{i,z}}p_{i,y}' & n_{q_{i,x}}p_{i,z}' - n_{q_{i,z}}p_{i,x}' & -n_{q_{i,x}}p_{i,y}' + n_{q_{i,y}}p_{i,x}' & n_{q_{i,x}} & n_{q_{i,y}} & n_{q_{i,z}} \end{bmatrix} \tag{6}$$

$$b_i = n_{q_{i,x}}(p_{i,x}' - q_{i,x}) + n_{q_{i,y}}(p_{i,y}' - q_{i,y}) + n_{q_{i,z}}(p_{i,z}' - q_{i,z}) \tag{7}$$

2.3 Optimization

To find the closed-form solution for the state vector $x = [\alpha, \beta, \gamma, t_x, t_y, t_z]^\top$, we need to minimize the sum of squared errors from our linearized cost function:

The objective is to minimize the linear least-squares problem:

$$\sum_{i=1}^{n} r_i^2(x) = \sum_{i=1}^{n} \|A_i x - b_i\|^2 \tag{8}$$

By stacking all the individual 1×6 matrices A_i and scalar b_i into a large $N \times 6$ matrix A and an $N \times 1$ vector b, the problem can be written in matrix form:

$$x_{opt} = \arg\min_x \|Ax - b\|^2 \tag{9}$$

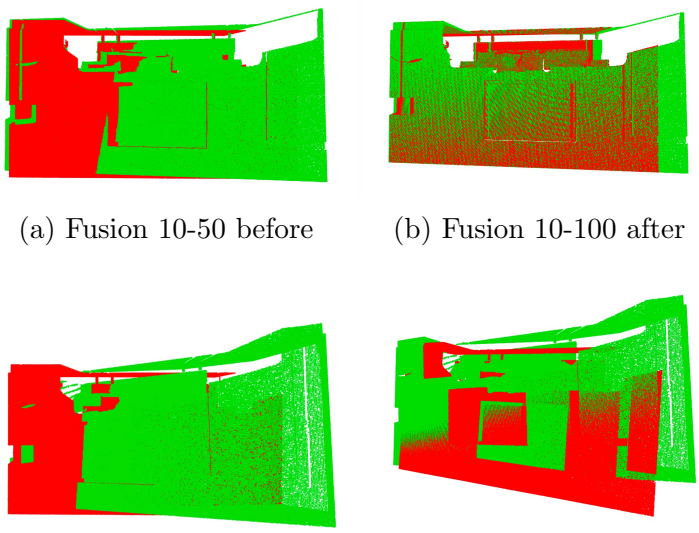
Taking the partial derivative with respect to x and setting it to 0 gives us the normal equation (a 6×6 linear system):

$$A^\top A x = A^\top b \tag{10}$$

From here, we can solve for x using **LU Formulation (6×6 system):** We can sum up n matrices to form the normal equation where $\sum(A_i^\top A_i)x = \sum(A_i^\top b_i)$. The closed-form solution using the pseudo-inverse is:

$$x = (A^\top A)^{-1} A^\top b \tag{11}$$

We compare two frame pairs: (10, 50) and (10, 100). When the two views are closer in time (10 and 50), ten ICP iterations suffice to align and merge them well (Figure 5). When they are farther apart (10 and 100), many more iterations are needed to obtain a reasonable alignment (Figure 6). In general, **a larger temporal gap between source and target requires more iterations** for the incremental rotations and translations to converge.



(a) Fusion 10-50 before (b) Fusion 10-100 after

(c) Fusion 10-100 before (d) Fusion 10-100 after

Figure 1: Visualization of ICP

3. Point-based Fusion

3.1 Filter

The `filter pass1` and `filter pass2` are implemented in `fusion.py`.

3.2 Merge

The `merge` operation updates existing points in the map by calculating a weighted average of the desired properties (positions, normals, and colors).

Given $p \in R^3$ in the map coordinate system with a weight w, and its corresponding point $q \in R^3$ in the frame's coordinate system with a weight 1, the transformation from the camera frame to the world frame is defined by the rotation matrix R_c^w and translation vector t_c^w.

Weighted average of the positions:

$$\bar{p} = \frac{wp + (R_c^w q + t_c^w)}{w + 1} \tag{12}$$

Weighted average of the normals:

$$\bar{n}_p = \frac{wn_p + R_c^w n_q}{w + 1} \tag{13}$$

3.3 Addition

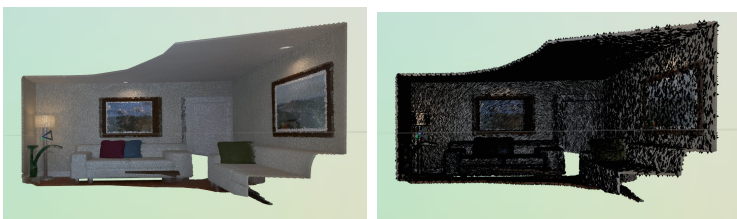
Implemented in `fusion.py`.

3.4 Results

The visualization of the fusion map with a normal map is shown in the figure.

After fusing 200 frames with spatial downsample factor 2, the map contains 2,498,299 points. Stacking 200 frames without merging would give $200 \times 81{,}600 = 16{,}320{,}000$ points.

Thus the compression ratio is $\frac{16{,}320{,}000}{2{,}498{,}299} \approx 6.53$.



(a) Fusion map (b) Normal map

Figure 2: Visualization of point-based fusion

4. Dense SLAM

4.1 Source & Target

In our pipeline, **the source is the map point cloud, and the target is the RGB-D frame** at that timestep, represented by its vertex map (and normal map) in the camera frame. We do not swap these roles: with frame-to-model registration, map points are projected into the current frame's range image to establish correspondences. **If we reversed source and target, we would need a regular vertex map for the entire map,** which we do not have.

4.2 Visualization of ICP + Fusion

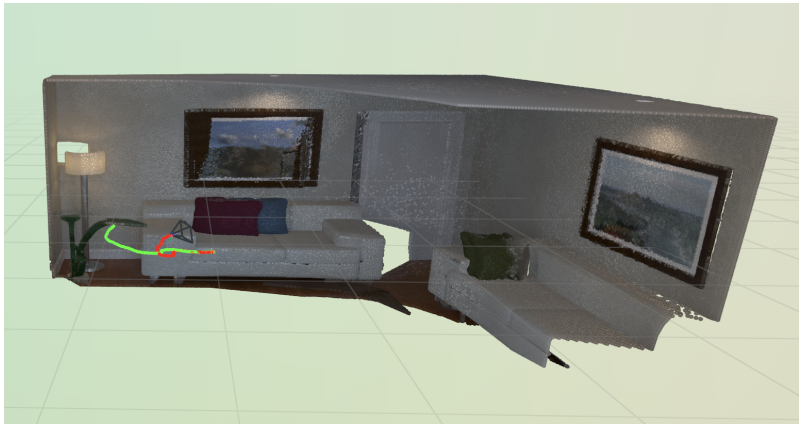


Figure 3: Fusion with poses estimated from frame-to-model ICP

4.3 Estimated Trajectory

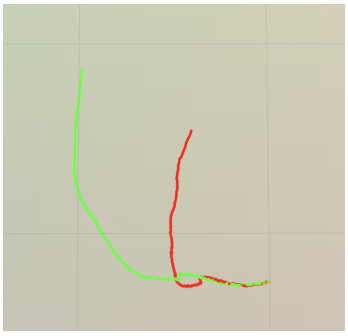
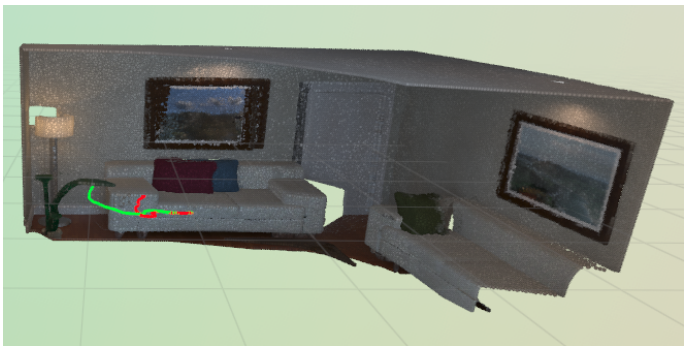


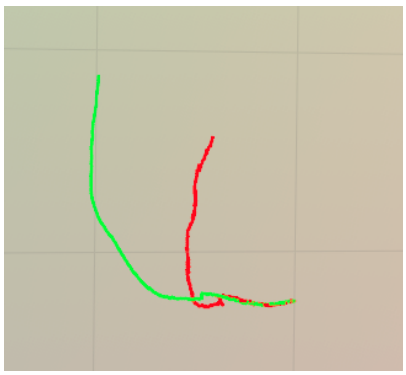
Figure 4: Estimated camera trajectory in red and GT in green
(mean abs trans error: 0.28m, max: 0.68m)

Bonus: Reducing Drift

We fuse frame-to-model and frame-to-frame ICP by selecting and scaling it with an adaptive gain.



(a) Updated fusion map



(b) Updated trajectory (mean error: 0.25m, max: 0.65m)

Figure 5: Visualization of updated fusion